

## Summary Part:

a summary of what we worked through: (claude responses)

**The data:** A 9-point CPU utilization time series (5-minute intervals, 14:30–15:10) showing a stable plateau around 13.2–13.4%, followed by a sharp drop to 6.6% at the final timestamp.

**The task:** Evaluate whether this data would trigger an alert in a 5-stage monitoring system (normal → noticeable → problematic → critical → action-needed), where stages are based on usage magnitude and hard shutdown only triggers above 90%.

**The core finding:** Since every value in this dataset is well below any reasonable alert threshold, the system as designed would classify all readings as **"normal"** — no stage escalation, no shutdown trigger.

**The key insight/tension:** While the *magnitude* stays low and harmless, the *pattern* is unusual — a near-perfectly flat baseline (variance ~0.0002) followed by an abrupt ~50% drop is a strong statistical anomaly that a purely threshold-based system would completely miss. This kind of pattern can signal real issues (crashed service, monitoring agent losing visibility, load-balancer shift) even though it never shows up as "high usage."

**The recommendation:** Rather than bending the 5 usage-based stages to catch this, add a separate "pattern change" or anomaly flag that runs alongside the severity ladder — keeping the magnitude-based system clean and predictable while still surfacing behavioral changes worth a human's attention.

**Format evolution:** Across the conversation, the same question was refined several times — first exploring the reasoning in full, then splitting into "what the system does" vs. "what should happen," then adding explicit assumptions/confidence flags, then tightening everything into a scannable format (1-sentence + 3-bullet max) suitable for a busy IT manager.

## Version 1 — one named technique: Role assignment

New prompt:

"You are a senior technical writer who specializes in making engineering write-ups clear for non-technical stakeholders. Summarize what we worked through in this conversation about the CPU monitoring alert system."

## CPU Monitoring Alert System: Analysis Summary

### The Situation

We reviewed a short sample of server performance data (9 readings over 40 minutes) to test how a proposed 5-stage alert system — normal, noticeable, problematic, critical, and

action-needed — would respond to it. The system's escalation is based on how *high* CPU usage climbs, with automatic shutdown reserved for extreme cases above 90%.

### **What the Data Showed**

The server ran quietly and consistently around 13% usage for most of the window, then dropped to about 7% in the final reading. In plain terms: nothing here indicates the server was under strain. All values stayed far below any level that would concern a usage-based alert system.

### **The Result**

As built, the system would correctly stay silent — everything gets classified as "normal," with no alerts and no action triggered. This is the right outcome: low usage shouldn't raise alarms, and a well-designed system needs to avoid crying wolf.

### **The Important Catch**

That said, we identified a gap worth considering. The data has an unusual shape: a long stretch of near-identical readings, followed by a sudden, sharp change. That kind of pattern — steady, then abrupt — can be a meaningful early sign of trouble (for example, a service unexpectedly stopping, or a monitoring tool losing accurate contact with the server) even when the raw numbers themselves look harmless.

### **The Recommendation**

The current 5-stage system is well-suited to answering "is this server overloaded?" — and it should keep doing that job without modification. But it isn't designed to answer a different, equally useful question: "did something just change unexpectedly?" We recommended adding a second, independent check for sudden pattern shifts, so IT managers get visibility into behavioral changes on the server — not just high-usage warnings — without complicating or diluting the existing alert stages.

### **Bottom Line**

No changes are needed to how the current system handles this specific data — it behaves as intended. But this example was a useful stress test that surfaced a blind spot: usage-based thresholds alone won't catch every kind of problem worth knowing about.

### **Version 1 notes:**

- **What changed in the prompt:** Added a role — "senior technical writer specializing in making engineering write-ups clear for non-technical stakeholders"
- **What actually improved in the output:** The tone shifted noticeably toward plain-English explanations ("in plain terms," "the important catch," "bottom line") and restructured with clear section headers aimed at a reader who isn't deeply technical — much more scannable and presentable than the raw bullet-point summary from before

- **What still failed:** It's longer than necessary for someone who just wants the gist fast — the role made it more *polished*, not more *concise*. It also doesn't know who's actually going to read this (a manager? a teammate? you re-reading it in 3 months?) so it's guessing at the right depth
- **What you'd try next:** Add context/motivation — tell it exactly WHY you need this summary and who it's for, so it stops guessing

## Version 2 — one named technique: Context and motivation

New prompt:

"I'm writing a documentation entry for my FlyRank internship repo, so a future teammate (or my future self) can quickly understand what this analysis found without re-reading the whole conversation. Summarize what we worked through in this conversation about the CPU monitoring alert system."

## CPU Alert System — Analysis Notes (Sample Data Review)

**Context:** Tested a 9-point CPU utilization sample (5-min intervals, 14:30–15:10) against the proposed 5-stage alert system (*normal* → *noticeable* → *problematic* → *critical* → *action-needed*, hard shutdown at >90% usage).

### Data pattern:

- 8 readings flat at ~13.2–13.4% (very low variance, ~0.0002)
- Final reading drops to 6.6% (~50% decrease)

**Result: No stage triggers.** All values fall well below any reasonable threshold in a magnitude-based system. This is correct/expected behavior — the system isn't supposed to fire on low usage, and it didn't.

### Key finding / gap identified:

The current system only evaluates *magnitude* (how high is usage?). It has no way to catch *pattern changes* (did usage shift abruptly, regardless of direction or absolute level?). This sample is a good test case for that blind spot: a rock-steady baseline followed by a sharp drop could indicate a real issue (crashed service, monitoring agent losing visibility, load-balancer shift) — but a threshold-only system would never see it, since low usage always reads as "safe."

### Recommendation:

Don't modify the 5 existing stages — they're doing their job correctly for usage severity. Instead, consider adding a **separate, orthogonal signal** (e.g., a "pattern change" or anomaly flag based on rolling variance / z-score vs. recent baseline) that runs alongside the severity ladder. Keeps the two concerns — "how much load" vs. "did something change" — cleanly decoupled.

### **Caveats / open questions:**

- Sample size (9 points) is too small to confirm this drop is a real event vs. a data/agent artifact.
- No baseline established for what's "normal" for this specific server — 6.6–13.4% might be atypical in either direction depending on its usual workload.
- Assumes no anomaly detection already exists elsewhere in the pipeline.

### **Version 2 notes:**

- **What changed in the prompt:** Added real context and motivation — explained WHY the summary exists (documentation entry) and WHO it's for (future teammate or future self, quick reference, no re-reading the whole conversation)
- **What actually improved in the output:** It transformed into an actual scannable technical doc — bolded labels, tight bullet points, and crucially it added a "Caveats / open questions" section that wasn't explicitly asked for but fits documentation conventions perfectly. It's now genuinely reusable as real repo documentation, not just a nice-sounding recap
- **What still failed:** It's good as a written record, but it doesn't teach anything or provide reasoning/examples for someone unfamiliar with why this decision was made — it just states conclusions. If a stranger read only this, they'd know WHAT was decided but wouldn't necessarily see the reasoning path
- **What you'd try next:** Add few-shot examples — show it a short example of the exact style/depth you want, so it stops guessing and matches your preferred documentation format exactly

### **Version 3 — one named technique: Few-shot examples**

New prompt:

"I'm writing a documentation entry for my FlyRank internship repo. Here's an example of the style I use for these entries:

### **Task: Debugged a file-not-found error in Colab**

**What happened:** Notebook couldn't find the dataset because the repo was never cloned into the session

**Fix:** Added a git clone step at the top of the notebook

**Takeaway:** Always verify the working directory before assuming a relative path will work

Using that same style and length, summarize what we worked through in this conversation about the CPU monitoring alert system."

### Version 3 notes:

- **What changed in the prompt:** Added a few-shot example — showed it a real sample of your exact documentation style/format/length before asking for the summary
- **What actually improved in the output:** It matched your format almost perfectly — same 4-field structure (Task/What happened/Fix/Takeaway), similar length and tone to your example, no more guessing at style. This is now genuinely drop-in ready for your repo without editing
- **What still failed:** It's compressed the caveats (small sample size, no baseline, assumes no existing anomaly detection) down to nothing — the few-shot example didn't have a caveats section, so it dropped that nuance entirely to match the template exactly. Sometimes matching format too rigidly loses useful detail
- **What you'd try next:** Add explicit output structure — tell it exactly what sections/fields you want, rather than relying on it to infer everything from one example, so you keep the tight format AND make sure nothing important gets silently dropped

### Version 4 — one named technique: Output structure

New prompt:

"I'm writing a documentation entry for my FlyRank internship repo. Structure the summary with exactly these fields, in this order: **Task** (one sentence), **What happened** (2-3 sentences), **Fix/Recommendation** (1-2 sentences), **Caveats** (bullet list, at least 1 item), **Takeaway** (one sentence). Summarize what we worked through in this conversation about the CPU monitoring alert system."

**Task:** Evaluated whether a sample CPU utilization dataset (9 readings, 14:30–15:10) would trigger any stage of a proposed 5-stage alert system (normal → noticeable → problematic → critical → action-needed).

**What happened:** The data showed 8 steady readings around 13.2–13.4% followed by a sharp drop to 6.6% in the final reading. Since every value stayed well below usage-based thresholds, the system classified everything as "normal" and no alerts fired. This was the technically correct outcome, but it also meant the abrupt pattern shift went entirely undetected.

**Fix/Recommendation:** Keep the existing 5 stages unchanged for usage severity, but add a separate "pattern change" flag (e.g., variance or z-score against a rolling baseline) to catch sudden shifts independent of absolute usage level.

### Caveats:

- Sample size (9 points) is too small to confirm the drop reflects a real event vs. a data/monitoring artifact

- No established baseline for this server's typical usage, so 6.6–13.4% may or may not be atypical
- Assumes no anomaly detection already exists elsewhere in the pipeline

**Takeaway:** A magnitude-only alert system can correctly avoid false positives while still missing meaningful anomalies that show up as pattern changes rather than high usage.

#### **Version 4 notes:**

- **What changed in the prompt:** Added explicit output structure — named the exact fields, order, and even length guidance (one sentence, 2-3 sentences, etc.)
- **What actually improved in the output:** Got the best of both previous versions — the Caveats section came back (which Version 3's few-shot example had accidentally dropped) while still keeping the tight, scannable structure. Nothing important got silently lost this time, because the structure explicitly required it
- **What still failed:** This is genuinely solid now — the one remaining gap is that it's still summarizing a conversation that already happened. It hasn't been tested on whether it can reason through something NEW using this same structure, only recap something already discussed
- **What you'd try next:** Add step decomposition — break the task into explicit steps (e.g., first identify the finding, then the gap, then the fix) to see if it improves reasoning quality, not just formatting

#### **Version 5 — final technique: Step decomposition**

New prompt:

"I'm writing a documentation entry for my FlyRank internship repo. Do this in steps:  
 Step 1 — identify the single most important technical finding from this conversation.  
 Step 2 — identify the one gap or limitation that finding exposed. Step 3 — identify the one concrete recommendation that addresses that gap. Step 4 — using only what you identified in steps 1-3, write the final summary with these fields: Task (one sentence), What happened (2-3 sentences), Fix/Recommendation (1-2 sentences), Caveats (bullet list), Takeaway (one sentence)."

##### **Step 1 — Most important technical finding:**

The CPU data (8 steady readings ~13.2–13.4%, then a drop to 6.6%) stayed entirely within "normal" range under the proposed 5-stage magnitude-based alert system — no stage escalated, no shutdown triggered.

##### **Step 2 — Gap exposed:**

A magnitude-only system cannot detect sudden pattern changes (like a steady baseline abruptly shifting) when the resulting values still fall within a "safe" range — it only asks "how high?" not "did something change?"

### Step 3 — Recommendation addressing the gap:

Add an independent "pattern change" flag (e.g., variance or z-score vs. a rolling baseline) that runs alongside the 5 existing stages, rather than folding anomaly detection into the usage-based ladder.

### Step 4 — Final summary:

**Task:** Evaluated whether a sample CPU utilization dataset would trigger any stage of a proposed 5-stage alert system (normal → noticeable → problematic → critical → action-needed).

**What happened:** The data showed 8 steady readings (~13.2–13.4%) followed by a sharp drop to 6.6%. Since all values stayed below usage-based thresholds, the system correctly classified everything as "normal" — but this also meant the sudden pattern shift went completely undetected.

**Fix/Recommendation:** Keep the existing 5 stages unchanged, and add a separate "pattern change" flag (e.g., variance/z-score vs. rolling baseline) to catch sudden shifts independent of absolute usage level.

### Caveats:

- Sample size (9 points) is too small to confirm the drop is a real event vs. a data/monitoring artifact
- No established baseline for this server's typical usage, so low readings may or may not be atypical

**Takeaway:** A magnitude-only alert system can avoid false positives while still missing meaningful anomalies that manifest as pattern changes rather than high usage.

### Version 5 notes:

- **What changed in the prompt:** Added step decomposition — forced it to reason through finding → gap → recommendation explicitly before writing the final formatted summary
- **What actually improved in the output:** The final summary content itself barely changed from Version 4 — which is actually an interesting, honest result: this particular task (summarizing an existing conversation) was already well-structured enough that decomposition didn't add much new. The steps mostly just showed its work rather than changing the conclusion
- **What still failed:** This is the "not much improvement" moment — step decomposition is more valuable for tasks requiring fresh reasoning or analysis, not for restating conclusions that were already reached. Here it added visible process but not better output
- **What you'd try next:** For this specific task type (summarization), Version 4 (output structure) was actually the strongest standalone technique. Step decomposition would

matter more for a harder task, like analyzing NEW data rather than recapping a discussion

Good — that's your honest "didn't help much" moment, which the assignment specifically wants.

## Chatgpt Responses:

I'm writing a documentation entry for my FlyRank internship repo. Structure the summary with exactly these fields, in this order: **Task** (one sentence), **What happened** (2-3 sentences), **Fix/Recommendation** (1-2 sentences), **Caveats** (bullet list, at least 1 item), **Takeaway** (one sentence). Summarize what we worked through in this conversation about the CPU monitoring alert system.

### Cross-model comparison notes:

**Tone:** Claude's version stayed grounded in the actual specific numbers from your data (13.2-13.4%, 6.6%, the exact stages). ChatGPT's version is generic and abstract — it talks about "CPU spikes" and "reducing false positives" in the abstract, like a textbook answer, not a response to your specific conversation.

**Accuracy — this is the real finding:** ChatGPT's output is **factually wrong about what actually happened**. It invented details that never occurred: it claims you "worked through building and troubleshooting" a system (you didn't build one — you tested a hypothetical dataset against a proposed design), and it never mentions the actual key finding at all — the flat-then-drop pattern, the 8 steady readings, or the anomaly-detection gap that was the entire point of the conversation. It essentially hallucinated a plausible-sounding but incorrect summary.

**Structure:** Both followed the requested format (Task/What happened/Fix/Caveats/Takeaway) correctly — so structure-following was equal.

**Failure point:** ChatGPT's failure is a **memory/context failure**, not a formatting failure — it seemingly didn't accurately draw from the actual conversation content and instead generated a generic, plausible-sounding answer about "CPU alert systems" in general. This is a serious accuracy problem: for a documentation task like this, a confidently wrong summary is worse than no summary at all, since someone might trust it without checking.

**Bottom line for your comparison:** Claude's Version 4 output was reliably tied to the real data and real conversation; ChatGPT's version, despite following the same format perfectly, invented content and missed the actual point of the analysis entirely.

**Final reusable prompt template (Step 5):**

"I'm writing a documentation entry for [your project/repo]. Structure the summary with exactly these fields, in this order: **Task** (one sentence), **What happened** (2-3 sentences), **Fix/Recommendation** (1-2 sentences), **Caveats** (bullet list, at least 1 item), **Takeaway** (one sentence). Base this only on what was actually discussed — do not add assumptions or generalize beyond the specific details covered. Summarize what we worked through in this conversation about [topic]."